

Lab-5 1A

Jibon Naher¹ and Samnun Azfar²

¹Lecturer, CSE

²Junior Lecturer, CSE

January 18, 2026

Part A (Mutex): Multi-Account Transfer System

Problem Statement

You are given three shared bank accounts with initial balances: $A = 1000$, $B = 1000$, $C = 1000$. Three threads run concurrently to perform transfers.

Input (Workload)

- **Thread T1:** Transfers 100 from $A \rightarrow B$ (10 times).
- **Thread T2:** Transfers 50 from $B \rightarrow C$ (20 times).
- **Thread T3:** Transfers 25 from $C \rightarrow A$ (40 times).

Constraints

- A transaction is one instance of money being transferred between accounts, that implies **Thread T1** executes 10 transactions.
- You can use **multiple mutexes**. One for each account.
- Transfers must be **all-or-nothing**.
- No account may ever go negative.

Expected Output

- **Intermediate:** Logs showing transfer start/end without interleaved errors. After each thread is done executing print the balances of the accounts.
- **Final State:** $A = 1000$, $B = 1000$, $C = 1000$. (Note: All transfers technically cancel out in sum).
- **Invariant:** $A + B + C = 3000$ at every step.

Part B (Semaphore): Limited-Capacity Transaction Server

Problem Statement

A transaction server can process at most 3 transactions concurrently.

Input

- 6 client threads ($C_1 \dots C_6$) arriving almost simultaneously.
- Each client requests access, holds it for 1 second, then leaves.

Constraints

- Use **one counting semaphore** initialized to 3.

Expected Output A line showing 6 boolean values where `C1=True` indicates C1 is being served. Use a mutex, a thread to monitor and a boolean array to track the status of the threads being served. Use delay functions to make the console printing readable.

```
C1=True, C2=True, C3=True, C4=False, C5=False, C6=False
C1=False, C2=True, C3=True, C4=True, C5=False, C6=False
```

Delay Functions

To simulate lengthy service times in concurrent threads, explicit delay functions are introduced so that a thread occupies the server for a realistic duration before releasing it.

On Linux systems, sleep-based system calls are commonly used. The required headers and examples are shown below.

```
#include <unistd.h>    // sleep(), usleep()
#include <time.h>       // nanosleep()
```

Basic delays can be implemented as:

```
sleep(2);                // delay for 2 seconds
usleep(500000);          // delay for 500 milliseconds
```

For higher precision, `nanosleep()` provides finer control:

```
struct timespec ts;
ts.tv_sec = 1;
ts.tv_nsec = 200000000; // 200 ms
nanosleep(&ts, NULL);
```

These delay mechanisms enable realistic modeling of service duration, contention, and waiting behavior in multi-threaded simulations.